Designing a Secure System Architecture For PayLink Africa

Fundamentals of Security Architecture

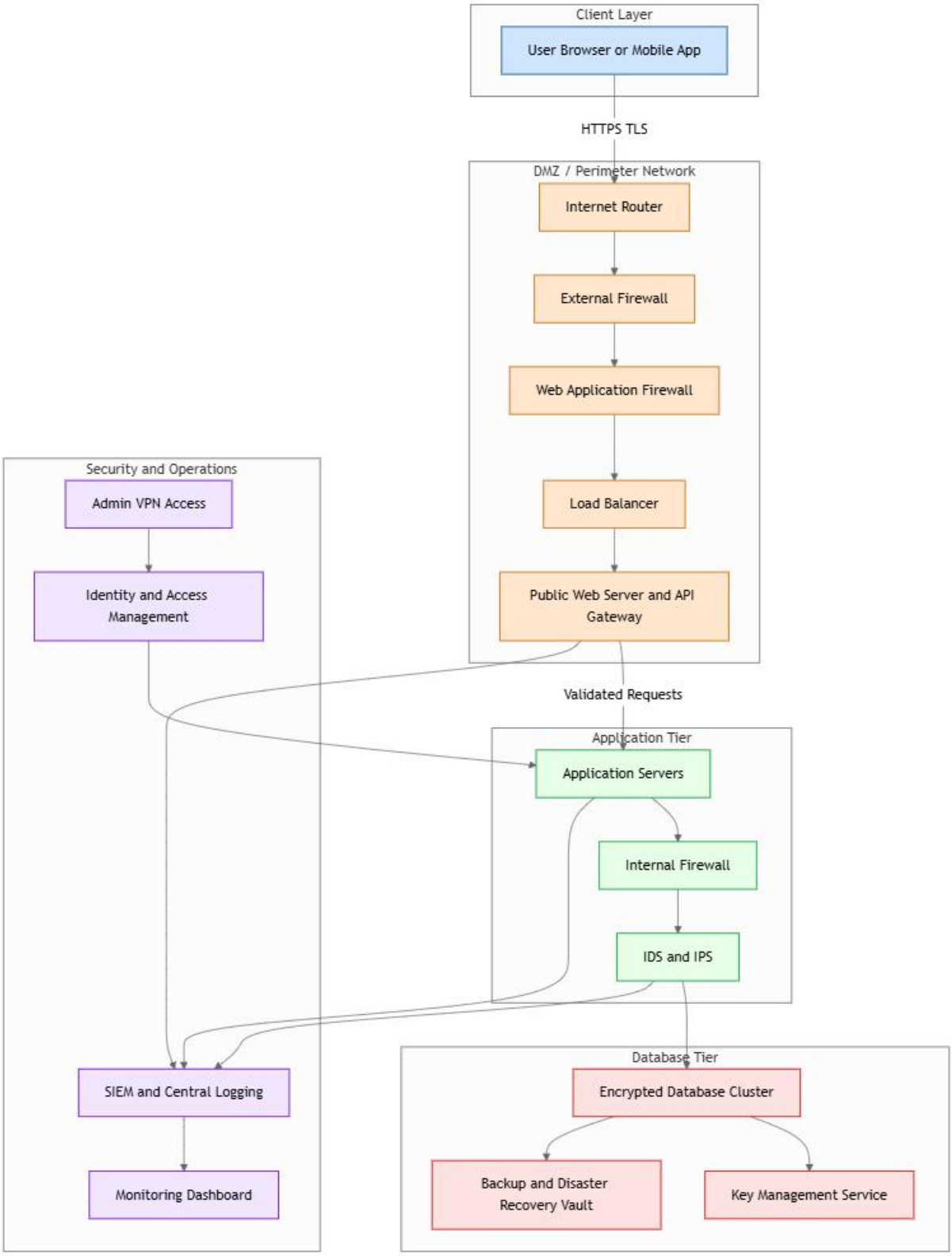
Name: Abeeb Abidoye

Student ID: ALT/SOE/025/3237

Date: 15th March 2026

ARCHITECTURE DESIGN

This document outlines the secure system architecture designed for PayLink Africa. The primary goal of this design is to protect sensitive financial data while making sure the platform remains scalable and compliant with industry regulations.



How the Architecture Flows

To provide a clear picture of how data travels securely, the architecture is broken down into distinct, specialized layers:

The Client Layer: External users interact with the system using a web browser or mobile app. All of these connections are locked down using HTTPS with TLS 1.3 encryption.

The Perimeter (DMZ): When traffic arrives, it first passes through an internet router and an external firewall. From there, it is filtered by a Web Application Firewall (WAF) and distributed by a load balancer before finally reaching the public web servers and API gateway. This layer acts as the frontline defense, utilizing DDoS protection, IDS/IPS monitoring, and strict API rate limiting to keep bad actors out.

The Application Tier: Deep inside the internal network, application servers do the heavy lifting for payment processing, wallet services, and authentication. These internal microservices securely talk to one another using mutual TLS (mTLS). Access here is tightly controlled by internal firewalls and robust Identity and Access Management (IAM).

The Database Tier: This is a highly restricted secure zone that houses the encrypted relational database containing customer and transaction records. It is protected by its own database firewall and is only accessible by the application servers. To ensure data safety, this tier also includes a Key Management Service (KMS) and a backup server for disaster recovery.

Security & Management Layer: Overseeing the entire operation is a centralized management layer. This includes a SIEM for continuous log monitoring, a SOC dashboard, vulnerability scanners, and patch management servers.

Controls Placement:

- **Access Control**: IAM, RBAC, MFA, API keys, OAuth2
- **Encryption in Transit**: TLS 1.3, mTLS
- **Encryption at Rest**: Database encryption, KMS
- **Backup/DR**: Daily snapshots, offsite replication
- **Monitoring**: SIEM, IDS/IPS, WAF logs
- **Risk Management**: Segmentation, least privilege, threat modeling, patching

EXPLANATION & JUSTIFICATION

The proposed architecture for PayLink Africa is designed to protect sensitive financial data while supporting scalability and regulatory compliance. The system follows the principles of the CIA triad, Confidentiality, Integrity, and Availability through layered controls and structured network segmentation.

Confidentiality

Keeping things under wraps mostly boils down to encryption, strict access controls, and keeping different parts of the network separated. For instance, whenever a user connects to the server, TLS 1.3 locks down that connection so nobody can snoop on payment details or authentication tokens. Behind the scenes, internal microservices use mutual TLS (mTLS) to communicate, which stops intruders from freely hopping around the network if they manage to get inside. We also keep the database tucked away in a highly secure zone that only the application servers can reach, with all stored data scrambled using a central Key Management Service. Finally, to keep the administrative side safe, staff must use multi-factor authentication and role-based access. Combined, these steps keep customer information strictly out of the wrong hands.

Integrity

To make sure data stays accurate and untouched, the setup relies on a mix of secure APIs, hashing, digital signatures, and tight change controls. When an API request comes in, the gateway acts like a bouncer-checking signatures, enforcing rate limits, and making sure the request format is actually valid. On the database side, things like transaction logs, automated backups, and built-in constraints prevent data corruption. Plus, a SIEM is always watching the logs for any weird behavior that might point to tampering. Keeping systems patched and sticking to configuration baselines also closes the door on known vulnerabilities. All of this guarantees that if someone tries to alter data, it won't go unnoticed.

Availability

For a payment platform, going offline isn't an option, so availability is built around having fail-safes and redundancy for everything. A load balancer actively spreads out incoming traffic so no single web server gets overwhelmed, and the application servers run in clusters ready to take over if one fails. The database constantly copies itself to a secondary site just in case we need to recover from a sudden crash. At the front door, DDoS protection stops malicious traffic spikes from taking the system down. Pair that with regular backups and solid disaster recovery plans, and the platform can bounce back fast from an outage , keeping payments flowing around the clock.

Defense in Depth

The architecture applies defense in depth by layering security controls across every tier. The DMZ includes a firewall, WAF, and IDS/IPS to filter and inspect traffic before it reaches internal systems. The application tier uses IAM, segmentation, and secure coding practices to protect business logic. The database tier adds encryption, access restrictions, and monitoring. Even if one layer is compromised, additional layers prevent full system compromise. This multi-layered approach is important for a fintech environment where attackers often target multiple points simultaneously.

Least Privilege

Least privilege is implemented through the RBAC, network segmentation, and API-level access restrictions. Employees receive only the permissions necessary for their roles, and service accounts are limited to specific functions. Application servers can query only the database tables required for their operations. Firewalls enforce strict allow-lists, and microservices authenticate each other using certificates. This minimizes the scale of any compromise.

Threats and Mitigations

1. SQL Injection
Mitigation: WAF filtering, parameterized queries, ORM frameworks, and database firewall rules. The API gateway validates input before it reaches the application tier.

2. DDoS Attacks
Mitigation: Cloud-based DDoS protection, rate limiting, load balancing, and auto-scaling. These controls ensure the platform remains available even under heavy attack.

3. Credential Theft / Account Takeover
Mitigation: MFA, device fingerprinting, anomaly detection, and encrypted credential storage. IAM policies enforce strong password requirements and session timeouts.

Framework Influence

The design is influenced by **NIST SP 800-53** and **ISO 27001**. NIST guided the selection of access control, audit logging, and system hardening controls, while ISO 27001 informed risk management, incident response, and governance practices. The layered architecture also aligns with SABSA's principle of mapping business requirements to security controls.

Overall, the architecture provides a robust, scalable, and compliant foundation for PayLink Africa's digital payment ecosystem.

REFLECTION

1. Hardest Part of the Architecture

The most challenging aspect of designing the architecture was balancing strong security controls with the performance and usability requirements of a fintech platform. Payment systems must be fast and responsive, yet they also handle highly sensitive data that requires strict protection. Implementing defense in depth without introducing excessive latency required careful placement of controls such as WAF, IDS/IPS, and encryption. Another difficulty was ensuring that least-privilege access did not hinder legitimate operational needs, especially for developers and support teams. Achieving the right balance between segmentation, monitoring, and operational efficiency required thoughtful trade-offs and alignment with industry frameworks.

2. How the Design Would Differ in a Full Cloud Deployment

If PayLink Africa operated entirely in the cloud, the architecture would rely heavily on native cloud security services. In such a case, instead of physical firewalls, cloud-based security groups, network ACLs, and managed WAF services would be used. IAM would be integrated with the cloud provider's identity platform, and encryption keys would be managed through a cloud KMS. Auto-scaling groups would replace manual server provisioning, and serverless components could reduce the attack surface.
Logging and monitoring would be streamlined, thanks to services such as AWS CloudWatch, Azure Monitor, or Google Cloud Logging. Multi-region replication would simplify disaster recovery. The design, in general, would become more adaptable, automated, and economica l.

3. Lessons About Balancing Security, Cost, and Usability

The key lesson is that security architecture is always a balancing act. Strong security controls are essential, but they must not degrade user experience or make the system too expensive to operate and manage. Over-engineering can lead to unnecessary complexity, while under-engineering exposes the organization to risk. The most effective approach is risk-based, invest heavily in controls that protect critical assets while using automation and cloud services to reduce operational costs. Usability must also be considered, because users will bypass controls that are too complicated. The goal is to create a secure system that remains intuitive, scalable, and financially sustainable.